

অপারেটিং সিস্টেমের ভূমিকা
(Introduction to Operating Systems)
প্রফেসর চেস্টার রেবেরো(Prof. Chester Rebeiro)
কম্পিউটার বিজ্ঞান ও প্রকৌশল বিভাগ
(Department of Computer Science and Engineering),
ইন্ডিয়ান ইনস্টিটিউট অফ টেকনোলজি,মাদ্রাজ
(Indian Institute of Technology, Madras)

সপ্তাহ – 07

বক্তৃতা – 34

থ্রেড – লাইট ওয়েট প্রসেস (Thread – Light Weight Process) Part 2

আমরা দেখেছি কিভাবে thread তৈরি করা হয়েছে এবং কিভাবে তারা parallelization দ্বারা বড় কাজ সমাধান করতে ব্যবহৃত হতে পারে। এখন, কিভাবে thread সিস্টেমের মধ্যে পরিচালিত হয় দেখা হবে। মূলত, আমরা thread context চালানো দেখেছি এবং সেইজন্য, আমরা thread resource গুলির পরিচালনা করে এমন কিছু entity প্রয়োজন, CPU তে কি thread চালানো উচিত, কি CPU ব্যবহার করা উচিত, ইত্যাদি।

(স্লাইড সময় : 00:45 দেখুন)

তাই এটি করার জন্য যে দুটি কৌশল উপলব্ধ রয়েছে তা user thread এবং kernel thread নামে পরিচিত। সুতরাং, user thread গুলি user level thread library দ্বারা সম্পন্ন হয়। তাই সাধারণত এই, kernel thread চলমান thread সম্পর্কে কিছুই জানেন না। Kernel threads -এ thread গুলি সরাসরি kernel গুলির দ্বারা সমর্থিত এবং এইগুলিকে কখনও কখনও light weight process হিসাবেও পরিচিত করা হয়। তাই, আমরা user thread এবং kernel thread গুলি আরও বিস্তারিতভাবে দেখব।

(স্লাইড সময় : 01:20 দেখুন)

সুতরাং আসুন শুরু করা যাক user thread দিয়ে। সুতরাং একটি user thread হিসাবে, যেমন এখানে দেখানো হয়েছে অপারেটিং সিস্টেম বা kernel এর execution এর জন্য kernel space আছে এবং user space আছে

যেখানে প্রসেসগুলি execute করে। এখন, এই প্রসেসের একাধিক thread চলমান হবে। তাই এই ছাড়াও এখানে আমাদের এখানে একটি run time system রয়েছে এখানে এই আয়তক্ষেত্র, যা এই বিশেষ প্রক্রিয়ার বিভিন্ন thread পরিচালনা করে। এছাড়াও কি প্রয়োজন হয় একটি thread table যা এই run time system এর অংশ হিসাবে সংরক্ষিত হয়। সুতরাং, এই thread table এই বিশেষ প্রক্রিয়ার শুধুমাত্র স্থানীয় thread গুলির তথ্য রয়েছে। সুতরাং অন্য একটি প্রক্রিয়ার জন্য এটির নিজস্ব run time system এবং নিজস্ব thread table থাকবে। সুতরাং, লক্ষ্য করুন যে thread table এর entry এর সংখ্যা executing thread এর সমান হয়। এছাড়াও লক্ষ্য করুন যে thread table যা TCB বা Thread Control Block হিসাবে পরিচিত সেটি প্রক্রিয়াটির Process control Block থেকে ভিন্ন যা kernel space এ সংরক্ষিত থাকে। মূলত thread table এ kernel এর process control block তুলনায় কম entry থাকে, এটি একটি user space এ রয়েছে। সুতরাং, user thread গুলির সুবিধা হলো এটি অত্যন্ত দ্রুত, এবং এটি সত্যিই light weight। মূলত এই কারণ thread পরিচালনার জন্য কোনও system call নেই। এখানে থাকা run time system টি সব thread management , এবং এর মধ্যে সমস্ত প্রকারের switching করে, ইত্যাদি। যেমন কার্নেলের জানা নেই যে একটি নির্দিষ্ট প্রক্রিয়ায় একাধিক thread আছে। সুতরাং, দ্বিতীয় সুবিধা যা আমরা দেখতে পাব যে এই সিস্টেমে thread সমর্থন করার এই বিশেষ প্রক্রিয়া অপারেটিং সিস্টেমের জন্য উপযোগী যা threading সমর্থন করে না। এই user thread এর অন্য গুরুত্বপূর্ণ সুবিধা হল যে thread গুলির মধ্যে switching অত্যন্ত দ্রুত, এবং এটি দ্রুত কেন তা হল, user mode থেকে protected mode এবং user mode এ পুনরায় ফিরে আসার জন্য কোনও switch নেই । সুতরাং অন্যদিকে, thread গুলির মধ্যে switch করার জন্য user mode এ একটি thread থেকে অন্য thread switching প্রয়োজন। এছাড়া অনেক অসুবিধাও আছে । একটি গুরুত্বপূর্ণ অসুবিধা হল অপারেটিং সিস্টেম kernel এবং thread গুলির মধ্যে সমন্বয়ের অভাব রয়েছে। তাই, এই কারণেই OS জানে না যে একটি প্রক্রিয়া বহু thread করা হয়, এবং কোনও নির্দিষ্ট পদ্ধতিতে কতগুলি thread উপস্থিত রয়েছে তা কোনও ইঙ্গিতও নেই। তাই কি সমস্যা হতে পারে এর একটি উদাহরণ দেখা যাক। সুতরাং, দুটি পদ্ধতির একটি সিস্টেম বিবেচনা করুন, এবং এই প্রক্রিয়াগুলির একটিতে 100 thread আছে। এখন kernel এই সম্পর্কে জানে না কারণ kernel একটি প্রক্রিয়াতে উপস্থিত বিভিন্ন thread সম্পর্কে জানে না এবং সেইজন্য, যখন context switch করা হয় তখন এটি জ্ঞাত নয় যে অন্য প্রসেসের তুলনায় একটি প্রসেসের অনেক thread এর কিছু প্রয়োজন। সুতরাং, এটি 100 thread প্রক্রিয়ার জন্য এবং একক থ্রেড দিয়ে প্রক্রিয়ার জন্য একই slice interval বরাদ্দ করবে। সুতরাং, কি হয় যে, যেহেতু কার্নেলের scheduler এভাবে চালানো thread সংখ্যা সম্পর্কে অনবগত কারণ, scheduling decision বেশি সংখ্যা thread এর সঙ্গে প্রসারিত করা যাবে না। সুতরাং, scheduling এর আরেকটি অসুবিধা আসে, যখন thread বিভিন্ন state এ থাকে। উদাহরণস্বরূপ, আসুন আমরা এই নির্দিষ্ট প্রক্রিয়ার তিনটি thread বলি, এবং এই thread গুলির মধ্যে একটি runnable state এ রয়েছে, অন্য দুটি thread blocked state এ রয়েছে। এখন অপারেটিং সিস্টেম এটি জানে না।

সুতরাং, সিদ্ধান্ত কি হওয়া উচিত? সুতরাং, যেহেতু thread গুলির মধ্যে একটি runnable হতে পারে এটি প্রসেসরে execute করতে পারে বা এটি একটি runnable প্রক্রিয়া হিসেবে বিবেচনা করা উচিত, অথবা যেহেতু দুটি thread রয়েছে যা blocked রয়েছে, এই প্রক্রিয়াটিকে একটি blocked করা প্রক্রিয়া হিসেবে বিবেচনা করা উচিত এবং এটি প্রসেসর মধ্যে চলবে না। একটি তৃতীয় বিষয় সংশ্লিষ্ট system call গুলির সাথে ঘটে, এখন অনুমান করা হয় যে system call গুলি block করা হয় যখন এই thread গুলির মধ্যে একটি system call invoke করে, তখন অন্য thread গুলিকে অপেক্ষা করতে হবে যতক্ষণ না প্রথম thread টি system call invocation সমাপ্ত করে। এই user level thread মডেল সমর্থন করার জন্য, কি প্রয়োজন হয় যে অপারেটিং সিস্টেমের সর্বদা non blocking system call সমর্থন করা উচিত।

(স্লাইড সময় : 06:35 দেখুন)

সুতরাং এখন আমরা kernel level thread গুলি দেখি। অতএব, যেহেতু user space এ চালানোর প্রক্রিয়াটি প্রতিটি thread এর সাথে একাধিক thread কে পৃথক বা একটি স্বাধীন execution unit হিসেবে দেখাতে পারে এবং এই thread resource এর ব্যবস্থাপনা kernel space এ করা হয়। সুতরাং, process control block ব্লক সহ যা এখানে একটি process table এ দেখানো হয়, কার্নেলটি kernel space এ একটি thread control block বজায় রাখে। সুতরাং, এটা user thread এর মত নয়, যেখানে user space এ thread table এর রক্ষণাবেক্ষণ করা হয়, এখানে thread table থ্রেড টেবিল বা TCB, thread control block kernel space এ রক্ষণাবেক্ষণ করা হয়। এবং সেইজন্য, কার্নেল একটি প্রসেসে সংশ্লিষ্ট thread সংখ্যা সম্পর্কে সচেতন, এবং সেইজন্য, এই বাস্তবতার উপর ভিত্তি করে অনেক সিদ্ধান্ত করতে পারে। উদাহরণ স্বরূপ; scheduler চলমান thread গুলির সংখ্যা অনুসারে, কতটা time slice প্রক্রিয়ায় বরাদ্দ করা উচিত সে বিষয়ে আরও স্মার্ট সিদ্ধান্ত নিতে পারে। উদাহরণস্বরূপ, scheduler একটি বড় সংখ্যক thread এর প্রসেসের ক্ষেত্রে বেশি time slice দিতে পারে। আরেকটি সুবিধা হল যে thread kernel দ্বারা পরিচালিত হয়, তাই blocking system call এর প্রয়োজন নেই। মূলত যখন একটি thread একটি block system call execute করে প্রক্রিয়ার অন্যান্য thread তাদের execution চালিয়ে যেতে পারে। এই বিশেষ kernel level thread model এর দুর্বলতা এটি ধীর। মূলত thread পরিচালনা kernel invocation করে, এবং সেইজন্য; যেহেতু এই কার্নেল invocations system call জড়িত তাই, এটি যথেষ্ট ধীর। অপারেটিং সিস্টেমের সাথেও রয়েছে overheads এবং OS এর কারণে কার্নেলের জন্য thread scheduling এবং প্রক্রিয়া scheduling পরিচালনা করা প্রয়োজন। সুতরাং, এর অর্থ হল কার্নেলের প্রসেস সম্পর্কে উপস্থিত metadata ছাড়াও, প্রতিটি thread এর জন্য আরো metadata উপস্থিত থাকা প্রয়োজন যার জন্য কার্নেলের overhead উল্লেখযোগ্য হয়। সুতরাং, যখন আমরা একটি অপারেটিং সিস্টেম ডিজাইন করি বা thread

সমর্থন করে এমন একটি সম্পূর্ণ সিস্টেম, এমন অনেক দিক আছে যা যন্ত্র নেওয়ার প্রয়োজন হতে পারে যা জটিলতা অনেকগুলি হতে পারে। সুতরাং এই বিশেষ স্লাইড এই বিষয়গুলির কিছু তুলে ধরে।

(স্লাইড সময় : 09:15 দেখুন)

উদাহরণস্বরূপ, যখন thread fork করে তখন সিস্টেমটির কি করা উচিত? সুতরাং, fork একটি system call যা অপারেটিং সিস্টেম চালানোর কারণ এবং ফলে প্রক্রিয়া পুনরাবৃত্তি হয়। সুতরাং, একটি নতুন প্রক্রিয়া তৈরি করা হবে যা উদ্ভাবন প্রক্রিয়ার একটি সঠিক কপি। এখন একটি thread system call হলে কি ঘটবে? সুতরাং, কয়েকটি বিকল্প আছে যা চিন্তা করতে পারে। সুতরাং, OS কি করবে? এই প্রক্রিয়ার মধ্যে সম্ভাবিত সমস্ত thread duplicate করা উচিত, অন্য কথায়, একটি নতুন প্রক্রিয়া তৈরি করা উচিত, একই পর্যায়ে একই সংখ্যার thread নিয়ে। সুতরাং, এটি করার তুলনায় বলা সহজ, মূলত এটি অনেক synchronization সমস্যা তৈরি করতে পারে। উদাহরণস্বরূপ, বিবেচনা করুন যে আমাদের দুটি থ্রেড দিয়ে একটি প্রক্রিয়া আছে, একটি থ্রেড একটি জটিল অপারেশনকে একটি critical section এ চালানো হচ্ছে, যা একটি critical resource access করছে এবং অন্য thread fork system call invoke করে। সুতরাং, কি হবে যদি অপারেটিং সিস্টেম সম্পূর্ণ প্রক্রিয়া তার সমস্ত থ্রেড এর সঙ্গে অনুরূপ করে? সুতরাং, আমরা নতুন প্রসেসের দ্বিতীয় থ্রেডটি উল্লেখ করে critical section এও অংশ নেব এবং আমরা আগের ভিডিওগুলিতে দেখেছি যে এটি বিপর্যয়কর হতে পারে, মূলত এটি প্রোগ্রামটির আউটপুটকে পরিবর্তন করতে পারে। এই নির্দিষ্ট দৃষ্টিভঙ্গি নকশা বা পরিচালনার সময় একটি অনুসরণ করতে পারে এমন আরেকটি পদ্ধতি হল caller thread কোথায় হবে? সুতরাং, যদিও প্রক্রিয়াটিতে 10 টি ভিন্ন থ্রেড থাকতে পারে, তবে এই থ্রেডগুলির মধ্যে একটি fork system call আহ্বান করে, তৈরি করা নতুন প্রক্রিয়াটি কেবল সেই থ্রেডটি ডুপ্লিকেট করবে। সুতরাং, তৈরি নতুন প্রক্রিয়ায় বাকি 9 টি থ্রেড থাকবে না, কিন্তু শুধুমাত্র এই একটি থ্রেড আছে। সুতরাং, যখন একটি থ্রেডের মধ্যে একটি segmentation fault হয় তখন কি ঘটবে? সুতরাং, অপারেটিং সিস্টেম কেবল একক থ্রেড বন্ধ করা উচিত কি, না সম্পূর্ণ প্রক্রিয়া বন্ধ করা উচিত? এখন এইগুলির জন্য পছন্দ করা সহজ নয় এবং অপারেটিং সিস্টেম designer দের অপারেটিং সিস্টেম design করার সময় কীভাবে এই দিকগুলি পরিচালনা করা যায় সে সম্পর্কে গুরুত্বপূর্ণ নির্বাচন করতে হবে।

(স্লাইড সময় : 11:46 দেখুন)

সুতরাং আসুন আমরা থ্রেড ব্যবহারের একটি সাধারণ অ্যাপ্লিকেশন দেখি। সুতরাং, আপনি উদাহরণস্বরূপ, একটি network card নিতে পারেন যেখানে packet গুলি network এ প্রবেশ করে এবং এই packet গুলিকে service করতে হবে থ্রেডগুলির মাধ্যমে। সুতরাং আসুন আমরা বলি আমাদের এখানে একটি লুপ আছে, উদাহরণস্বরূপ একটি ইভেন্টের জন্য অপেক্ষা করে, network packet এ যখন কোনও ঘটনা ঘটে তখন এটি spawn করে বা এটি একটি থ্রেড তৈরি করে যা ইভেন্টটি পরিবেশন করে এবং তারপর সমাপ্ত হয়। সুতরাং, যে ইভেন্টের servicing

সময়, অন্য ইভেন্ট ঘটে, তাহলে, একটি নতুন থ্রেড তৈরি করা হয়। এই ভাবে একযোগে বিভিন্ন ঘটনা পর পর servicing করা হয়। তাই এই পদ্ধতিটি scalable কারণ এটি একযোগে একাধিক ইভেন্ট service করতে পারে। সুতরাং, এই বিশেষ মডেলের অসুবিধা হল overhead । মূলত thread create এবং terminate যদিও কম ওভারহেড রয়েছে, তবুও এটি একটি ওভারহেড যা কার্যক্ষমতা হ্রাস করতে পারে এবং এটি সমগ্র সিস্টেমকে প্রভাবিত করতে পারে। সুতরাং, কি অ্যাপ্লিকেশন সাধারণত করে, Thread pool নামে পরিচিত technique ব্যবহার করা হয়।

(স্লাইড সময় : 13:10 দেখুন)

এই বিশেষ কৌশলে, creation এর উপর অ্যাপ্লিকেশনটি কি করে তা হল যে এটি একটি thread pool তৈরি করতে যাচ্ছে। উদাহরণস্বরূপ, এটি 50 অথবা 100 টি ভিন্ন থ্রেড তৈরি করতে পারে এবং এই থ্রেড সাধারণত একটি blocked state এ থাকবে। এখন, এমন একটি প্রধান লুপ থাকবে যা ইভেন্টের জন্য অপেক্ষা করে এবং যখন একটি ঘটনা ঘটে তখন এই থ্রেডগুলির মধ্যে একটি wake up হবে যা blocked state এ রয়েছে তখন থ্রেডটি ইভেন্টটি service করবে এবং blocked state এ ফিরে যাবে। সুতরাং, এই উপায়ে যদি এই পুলের মধ্যে 50 টি থ্রেড তৈরি করা হয় যা ইতিমধ্যে তৈরি করা হয়েছে তবে সেখানে 50 টি ইভেন্ট রয়েছে যা এক সাথে serviced হতে পারে কোণে overhead ছাড়াই। 51 তম ইভেন্টটি ঘটেছে যখন thread pool এর সমস্ত 50 টি থ্রেড ইভেন্টের servicing এ ব্যস্ত থাকে, তারপর 51 তম ইভেন্টটিকে অপেক্ষা করতে হবে। সুতরাং, এই পদ্ধতিতে আমরা দেখি যে যখনই কোন ঘটনা ঘটবে তখন আমরা থ্রেডগুলি তৈরি ও ধ্বংস করার জন্য overhead গুলি দূর করেছি। তাই এখন, শুধুমাত্র প্রয়োজন thread pool থেকে একটি থ্রেড বেছে নেওয়া যা পরে ইভেন্টটিকে service দেবে। এখন থ্রেড পুলের সাথে আরেকটি গুরুত্বপূর্ণ দিক হল পুলের থ্রেডের সংখ্যা। সুতরাং, এটি প্রতিটি অ্যাপ্লিকেশনের জন্য গুরুত্বপূর্ণ এবং খুব অ্যাপ্লিকেশন নির্ভরশীল হতে যাচ্ছে। উদাহরণস্বরূপ, যদি আপনার খুব কম সংখ্যক থ্রেড 4 টি বা 5 টি দিয়ে একটি থ্রেড পুল থাকে তবে, এটি একযোগে শুধুমাত্র 4 বা 5 টি ইভেন্ট service করতে পারে। যদি আরো ঘটনা ঘটতে থাকে তবে থ্রেডগুলি তাদের servicing শেষ না হওয়া পর্যন্ত ইভেন্টগুলি queued হতে হবে এবং কর্মক্ষমতা প্রভাবিত হবে। অন্যদিকে যদি আমাদের কাছে thread pool এর মধ্যে প্রচুর সংখ্যক থ্রেড হাজার হাজার থ্রেড থাকে যা কোনও event ঘটায় না, তাহলে প্রায়ই প্রচুর সংখ্যক থ্রেড thread pool এর মধ্যে resource ধ্বংস করে। অতএব, থ্রেড পুলের থ্রেডগুলির সংখ্যা অত্যন্ত গুরুত্বপূর্ণ যে অ্যাপ্লিকেশন ডিজাইনারকে তৈরি করতে হবে। সুতরাং, এর সাথে আমরা থ্রেডগুলির একটি সংক্ষিপ্ত ভূমিকা প্রদান করেছি, আমরা থ্রেড এবং প্রসেসের মধ্যে পার্থক্য দেখেছি এবং কিভাবে execution সময় কমানোর জন্য এবং অ্যাপ্লিকেশনগুলির কার্যকারিতা উন্নত করতে আসলে কীভাবে থ্রেড ব্যবহার করা যেতে পারে, এবং আমরা একটি সংক্ষিপ্ত ভূমিকা দেখেছি কিভাবে থ্রেড অপারেটিং সিস্টেম দ্বারা পরিচালিত হয় এবং থ্রেডের বিভিন্ন ব্যবহার দেখেছি। ধন্যবাদ।

